



Hautes Études des Sciences et Techniques de
l'Ingénierie et du Management

Projet de fin d'année

FILIÈRE : GÉNIE INFORMATIQUE ET INTELLIGENCE
ARTIFICIELLE

CONCEPTION ET DÉVELOPPEMENT D'UNE
PLATEFORME DE COMMANDE ET D'INTERACTION
AUTONOME
POUR UN ROBOT DE SERVICE ANDROID

Réalisé par :

Claudia LUSAMOTE
KIMFUTA
Ghassane KHALIFI

Encadrante pédagogique :

Mme MESKINI Fatime
Zahra

Classe : 1^{ÈRE} DU CYCLE D'INGÉNIEUR

Année universitaire : 2025 - 2026

REMERCIEMENTS

Nous tenons à exprimer notre profonde gratitude à notre encadrant, Dr. Sridath Tula, pour son accompagnement, sa disponibilité et ses précieux conseils tout au long de la réalisation de ce projet. Son expertise, ses orientations et son soutien constant ont grandement contribué à l'aboutissement de ce travail.

Nous adressons également nos sincères remerciements à l'ensemble du corps professoral ainsi qu'au personnel administratif et technique de HESTIM pour la qualité de la formation dispensée, leur professionnalisme et les moyens mis à notre disposition tout au long de notre parcours académique.

Nous remercions particulièrement les responsables de la filière Génie Informatique pour leur encadrement et les connaissances qu'ils nous ont transmises au cours de ces années d'études, nous permettant d'acquérir les compétences nécessaires à la réalisation de ce projet.

Nous souhaitons également exprimer notre reconnaissance envers toutes les personnes qui, de près ou de loin, ont contribué à la réussite de ce travail par leurs conseils, leurs encouragements ou leur soutien.

Enfin, nous adressons nos plus sincères remerciements à nos familles et à nos proches pour leur patience, leur confiance et leurs encouragements constants. Leur soutien moral a été une source de motivation essentielle tout au long de cette expérience.

Ces remerciements témoignent de notre reconnaissance envers toutes les personnes qui ont contribué à la réussite de ce projet et à l'enrichissement de notre parcours académique et personnel.

RÉSUMÉ

Les robots de service mobiles sont généralement livrés avec un écosystème logiciel **fermé** : application propriétaire, absence de documentation et de protocole de communication publié. Ce constat limite toute personnalisation par l'utilisateur final. Le projet **CYBEL**, mené dans le cadre d'un stage de fin d'année (PFA), vise à concevoir une **plateforme de commande et d'interaction indépendante** pour un robot de réception mobile *CLOT TY1251D-03195*.

La démarche adoptée repose sur une **rétro-ingénierie** incrémentale et non destructive du protocole de communication interne : balayage des services réseau exposés, introspection des topics et services **ROS** via *roslaunch/rostopic*, et vérification systématique de l'effectivité de chaque commande. Sur cette base, une **architecture en trois couches** a été conçue : un **SDK Python** proposant une implémentation simulée (*mock*) et réelle interchangeable, une **API FastAPI** (REST + WebSocket) et une **interface web** Vite/TypeScript.

À ce stade, la connectivité avec le robot est établie, le protocole de **télémétrie**, de **commande de vitesse** et de **navigation autonome** a été reconstruit et intégré. La principale difficulté concerne le canal de **synthèse vocale (TTS)**, non exposé par ROS et nécessitant un accès complémentaire au sous-système Android.

Mots-clés : robotique de service, rétro-ingénierie de protocole, ROS, roslaunch, FastAPI, interface homme-robot, navigation autonome, synthèse vocale.

ABSTRACT

Mobile service robots are typically delivered with a **closed** software ecosystem : proprietary application, undocumented communication protocol, and no public API. This situation prevents end users from customising the robot's behaviour to suit their specific needs. The **CYBEL** project, carried out as part of a final-year internship (PFA), aims to design an **independent command and interaction platform** for a mobile reception robot *CIOT TY1251D-03195*.

The approach relies on an incremental, non-destructive **reverse engineering** of the robot's internal communication protocol : scanning of exposed network services, introspection of **ROS** topics and services via *rosbridge/rosapi*, and systematic verification of each command's effectiveness. On this basis, a **three-layer architecture** was designed : a **Python SDK** offering interchangeable simulated (*mock*) and real implementations, a **FastAPI** backend (REST + WebSocket), and a **Vite/TypeScript** web interface.

At this stage, connectivity with the robot has been established, and the **telemetry**, **velocity control**, and **autonomous navigation** protocols have been reconstructed and integrated. The main remaining challenge concerns the **text-to-speech (TTS)** channel, which is not exposed through ROS and requires additional access to the Android subsystem.

Keywords : service robotics, protocol reverse engineering, ROS, rosbridge, FastAPI, human-robot interaction, autonomous navigation, text-to-speech.

Table des matières

REMERCIEMENTS	i
RÉSUMÉ	ii
ABSTRACT	iii
LISTE DES ABRÉVIATIONS	ix
INTRODUCTION GÉNÉRALE	1
1 Présentation de l'entreprise	2
1.1 Activité principale et services	2
1.2 Organisation interne	2
1.2.1 Organigramme de l'HESTIM	2
1.3 Positionnement dans le secteur	3
1.4 Valeurs et engagements	3
1.5 Lien avec notre projet	4
1.6 Conclusion	4
2 Analyse du besoin et étude préalable	5
2.1 Contexte du projet	5
2.2 Problématique	9
2.3 Objectifs du projet	10
2.3.1 Objectif général	10
2.3.2 Objectifs spécifiques	10
2.4 Cahier des charges	11
2.4.1 Périmètre du projet	11
2.4.2 Acteurs du système	12
2.4.3 Besoins fonctionnels	12
2.4.4 Besoins non fonctionnels	13
2.4.5 Contraintes du projet	14
2.4.6 Livrables attendus	14
2.4.7 Critères de réception et de validation	14
2.5 Solution proposée	15
2.6 Conclusion	15
3 État de l'art	16
3.1 Fondements théoriques	16
3.2 État des solutions existantes	16
3.3 Analyse comparative	16

3.4	Limites des solutions existantes	16
3.5	Synthèse	17
3.6	Conclusion	18
4	Analyse et conception du système	19
4.1	Analyse fonctionnelle	19
4.1.1	Identification des acteurs	19
4.1.2	Besoins fonctionnels	20
4.1.3	Besoins non fonctionnels	21
4.2	Architecture générale de CYBEL	22
4.3	Architecture logicielle	22
4.3.1	Couche d'accès au robot (SDK)	23
4.3.2	Couche API	23
4.3.3	Couche présentation	24
4.4	Diagramme des composants	24
4.5	Diagrammes de cas d'utilisation	25
4.6	Diagrammes de séquence	26
4.7	Modèle des flux de communication	26
4.7.1	Robot $\leftrightarrow$ rosbridge	26
4.7.2	rosbridge $\leftrightarrow$ Backend	27
4.7.3	Backend $\leftrightarrow$ Interfaces utilisateur	27
4.7.4	Backend $\leftrightarrow$ Applications Android	27
4.8	Conclusion	27
5	Méthodologie et planification	28
5.1	Mode de travail et organisation	28
5.2	Planning du projet	28
5.3	Outils, Langage et Technologie utilisés	30
5.3.1	ROS	30
5.3.2	FastAPI	30
5.3.3	TypeScript	30
5.3.4	Android	30
5.3.5	ADB	30
5.3.6	ROS	30
5.4	Découverte du Robot CIOT TY1251D-03195 "CYBEL"	30
5.5	Proposition de documentation technique	30
5.6	Apports du stage	30
5.7	Difficultés rencontrées durant la phase d'investigation	30
5.8	Conclusion	30

6	Réalisation et implémentation	31
6.1	Mise en place de l'environnement	31
6.2	Découverte et analyse du robot	31
6.3	Reverse Engineering du système	31
6.3.1	Analyse réseau	31
6.3.2	Identification des protocoles	31
6.3.3	Étude des commandes	31
6.4	Développement du backend CYBEL	31
6.5	Développement de l'application Android	31
6.6	Développement de l'interface Web	31
6.7	Mise en place du système TTS	31
6.8	Intégration des différents modules	31
6.9	Conclusion	31
7	Validation, résultats et analyse critique	32
7.1	Stratégie de validation	32
7.2	Scénarios de tests	32
7.3	Résultats obtenus	32
7.4	Indicateurs de performance	32
7.5	Contributions personnelles	32
7.6	Difficultés rencontrées et solutions apportées	32
7.7	Limites du projet	32
7.8	Perspectives d'amélioration	32
7.9	Conclusion	32
	CONCLUSION GÉNÉRALE	33
	BIBLIOGRAPHIE	34
	WEBOGRAPHIE	35
	ANNEXES	36
	Annexe 1 : Capture d'écran de la carte	36
	Annexe 1 : Capture d'écran de la terminale	36
	Annexe 2 : Commandes utilisées	36
	Annexe 3 : Code source (extrait)	37
	Annexe 4 : Diagrammes détaillés	38
	Annexe 5 : Documentation technique complémentaire	38

Table des figures

1	Organigramme de l'HESTIM	3
2	Robot CIOT TY1251D-03195 "CYBEL"	6
3	Architecture système globale du robot CIOT TY1251D-03195	8
4	Architecture générale du système CYBEL	23
5	Architecture en couches du système CYBEL	24
6	Diagramme de composants du système CYBEL	25
7	Diagramme de cas d'utilisation du système CYBEL	25
8	Diagramme de séquence de la navigation vers un point sélectionné	26
9	Diagramme de Gantt du projet CYBEL	29
10	Carte du laboratoire	36
11	Génération de clé SSH	37
12	Organigramme de l'entreprise	38
13	Organigramme de l'entreprise	38

Liste des tableaux

1	Périmètre du projet : éléments inclus et exclus	11
2	Acteurs du système et leurs rôles	12
3	Besoins fonctionnels du système	13
4	Besoins non fonctionnels du système	13
5	Critères de réception et de validation	15
6	Panorama des solutions existantes comparables	17
7	Benchmark comparatif des solutions	17
8	Besoins fonctionnels du système CYBEL, synthèse par fonctionnalité	21
9	Besoins non fonctionnels du système CYBEL, synthèse par critère	22

LISTE DES ABRÉVIATIONS

ADB	Android Debug Bridge
API	Application Programming Interface (Interface de programmation applicative)
APK	Android Package (format d'installation des applications Android)
CYBEL	Nom du projet de plateforme de commande et d'interaction pour le robot CIOT TY1251D-03195 ainsi nom commun du robot
DHCP	Dynamic Host Configuration Protocol
HRI	Human-Robot Interaction (Interaction Homme-Robot)
HTTP	HyperText Transfer Protocol
IHM	Interface Homme-Machine
IoT	Internet of Things (Internet des objets)
IP	Internet Protocol
JSON	JavaScript Object Notation
LiDAR	Light Detection and Ranging
MQTT	Message Queuing Telemetry Transport
OWASP	Open Web Application Security Project
PFA	Projet de Fin d'Année
REST	Representational State Transfer
RK3399	Référence du système sur puce (SoC) équipant la tête Android du robot
ROS	Robot Operating System
rosbridge	Composant ROS exposant les topics et services ROS via WebSocket
rosapi	Service ROS d'introspection des topics, services et types de messages
SDK	Software Development Kit (Kit de développement logiciel)
SLAM	Simultaneous Localization and Mapping (Cartographie et localisation simultanées)
SoC	System on Chip (Système sur puce)
TCP/IP	Transmission Control Protocol / Internet Protocol
TTS	Text-To-Speech (Synthèse vocale)
UML	Unified Modeling Language

UI User Interface (Interface utilisateur)

WiFi Wireless Fidelity

WebSocket Protocole de communication bidirectionnelle en temps réel sur TCP

INTRODUCTION GÉNÉRALE

Les robots de service autonomes occupent une place croissante dans les environnements professionnels : accueil, guidage, téléprésence grâce à la maturité des technologies de navigation autonome et d'interaction homme-robot. Ces robots sont cependant généralement livrés avec un écosystème logiciel fermé : application propriétaire, protocole non documenté, absence d'API publique. Cette fermeture limite fortement la capacité des utilisateurs à adapter leur comportement à des besoins spécifiques.

C'est dans ce contexte que s'inscrit le robot de réception mobile **CIOT TY1251D-03195**, mis à disposition par HESTIM Engineering & Business School dans le cadre d'un stage de fin d'année. Composé d'un châssis de navigation sous ROS et d'une tête Android assurant l'interface visiteur, ce robot dépend entièrement d'une application propriétaire pour laquelle aucune documentation n'est disponible. Le projet **CYBEL**, objet de ce rapport, vise à concevoir une plateforme logicielle indépendante permettant de superviser, piloter et faire interagir ce robot sans recourir à l'application d'origine.

La problématique du stage est donc double : comment établir une communication fiable avec un système robotique fermé dont ni le protocole, ni les points d'accès réseau ne sont connus, et comment construire à partir de cette compréhension reconstruite une solution de contrôle robuste et extensible ? Le stage, d'une durée de trois à quatre mois, poursuit ainsi deux objectifs : procéder à une rétro-ingénierie méthodique du protocole de communication interne du robot, puis développer sur cette base une plateforme de commande complète, incluant une interface opérateur, une interface visiteur et des fonctionnalités d'interaction tactile et vocale. La démarche adoptée repose sur une approche itérative et prudente, où chaque hypothèse sur le fonctionnement du robot est systématiquement vérifiée avant d'être considérée comme acquise.

Le présent rapport restitue cette démarche en sept chapitres : présentation de l'entreprise d'accueil, analyse du besoin et étude préalable, état de l'art, analyse et conception du système CYBEL, méthodologie et planification, réalisation et implémentation, puis validation, résultats et analyse critique, avant une conclusion générale dressant le bilan du stage et ses perspectives.

1 Présentation de l'entreprise

L'HESTIM Engineering School (Hautes Études des Sciences et Techniques de l'ingénierie et du Management) est un établissement privé d'enseignement supérieur basé à Casablanca, au Maroc. Fondée en 2006, elle forme chaque année plusieurs centaines d'étudiants dans les domaines de l'ingénierie et du management. HESTIM fonctionne comme une entreprise de formation et de services académiques, structurée pour répondre aux besoins croissants des secteurs industriels et technologiques au Maroc et à l'international.

1.1 Activité principale et services

HESTIM propose des formations d'ingénieurs et de managers dans plusieurs spécialités, notamment :

- Génie Informatique et Intelligence Artificielle
- Génie Industriel et Logistique
- Génie Civil et Travaux Publics
- Management et Administration des Affaires

L'établissement offre des programmes d'enseignement initial, de formation continue et de doubles diplômes, en partenariat avec des universités et écoles étrangères. Il met également à disposition des laboratoires techniques et des équipements modernes : robots collaboratifs, capteurs 3D, robots de service Android ainsi que des services de recherche appliquée pour accompagner l'innovation.

1.2 Organisation interne

HESTIM est organisée autour de plusieurs pôles clés :

- **Pôle pédagogique**, qui regroupe les départements de formation spécialisés.
- **Pôle administratif**, qui assure la gestion des étudiants et des relations avec les entreprises partenaires.
- **Pôle recherche et développement**, qui coordonne les projets innovants menés en collaboration avec des entreprises ou dans le cadre de projets étudiants, dont le projet CYBEL fait partie.

1.2.1 Organigramme de l'HESTIM

Pour illustrer la structure interne de l'HESTIM et mieux situer notre projet dans son environnement, voici un organigramme simplifié de l'établissement.

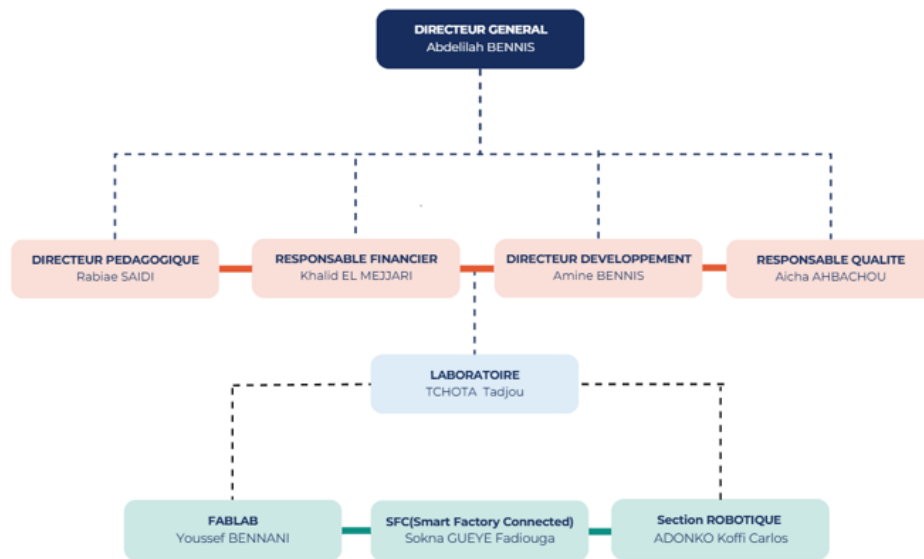


FIGURE 1 – Organigramme de l'HESTIM

1.3 Positionnement dans le secteur

HESTIM s'est imposée comme l'une des écoles d'ingénierie et de management les plus reconnues au Maroc. Elle bénéficie d'une réputation solide grâce à :

- la qualité de ses formations orientées vers les besoins concrets du marché ;
- ses nombreux partenariats avec des acteurs industriels nationaux et internationaux ;
- son engagement dans l'innovation pédagogique et technologique, notamment dans les domaines de la robotique et de l'intelligence artificielle.

HESTIM accueille un public varié, marocain et étranger, et cible des secteurs clés comme l'industrie, les technologies de l'information, le BTP et la logistique.

1.4 Valeurs et engagements

HESTIM place l'innovation, la qualité pédagogique et l'ouverture internationale au cœur de sa stratégie. L'école vise à former des ingénieurs et managers capables de répondre aux défis actuels et futurs de l'industrie, avec un fort accent sur l'éthique, l'esprit d'équipe et l'adaptabilité aux évolutions technologiques.

1.5 Lien avec notre projet

Nous avons réalisé le projet CYBEL au sein de HESTIM, considérée comme l'établissement d'accueil, car l'école offre un cadre idéal pour mettre en pratique nos connaissances en génie informatique et en intelligence artificielle. La présence sur site d'un robot de service Android *CIOT TY1251D-03195* nous a permis d'expérimenter directement sur le matériel cible. Grâce à ses équipements modernes et à son environnement de travail stimulant, nous avons pu développer des compétences techniques en rétro-ingénierie de protocoles, en développement d'API et en interfaces homme-robot, en parfaite adéquation avec le programme de la filière Génie Informatique.

1.6 Conclusion

La présentation de l'HESTIM Engineering School met en évidence un environnement académique et technologique favorable à la réalisation de projets innovants. Sa structure, ses équipements et en particulier la disponibilité du robot CYBEL et ses formations orientées vers l'industrie 4.0 nous ont permis de mener le projet CYBEL dans des conditions optimales, alliant rigueur académique et mise en situation professionnelle réelle.

2 Analyse du besoin et étude préalable

Ce chapitre présente le contexte général du projet CYBEL ainsi que l'analyse du besoin ayant conduit à la définition de la solution proposée. Il met en évidence l'architecture du système étudié, la problématique associée, les objectifs du projet ainsi que le cahier des charges fonctionnel et technique.

2.1 Contexte du projet

Présentation du robot CIOT TY1251D-03195

Le robot étudié est un **robot de réception mobile** commercialisé sous la référence **CIOT TY1251D-03195** illustré sur la figure 2. Il est composé de deux sous-systèmes matériels distincts, reliés en interne :

- un **châssis de navigation** fonctionnant sous **Linux embarqué avec ROS**, responsable de la localisation (SLAM), de la planification de trajectoire (`move_base`) et de l'exécution des commandes de mouvement ;
- un « **upper body** » **Android 7.1** (SoC RK3399, 2 Go de RAM, 16 Go de stockage), équipé d'un écran tactile 15,6" (1920×1080), de caméras (reconnaissance faciale, surveillance) et de haut-parleurs, qui héberge l'application propriétaire d'accueil et l'interface utilisateur standard du robot.

Le robot dispose d'un point d'accès WiFi propre (TY1251D-03195) permettant à un poste externe de rejoindre son réseau local. La topologie réseau s'est révélée plus complexe que ne le laissait supposer une simple architecture à deux hôtes : le châssis est joignable à l'adresse fixe 10.42.0.1, tandis que la tête Android obtient une adresse variable par DHCP sur un second segment 172.16.0.0/16 une première IP documentée (172.16.0.88) s'est révélée périmée en cours de projet. Les deux sous-systèmes sont par ailleurs reliés en interne par une liaison filaire `eth0` sur le segment 192.168.20.0/24 (192.168.20.1 pour la tête, 192.168.20.22 pour le châssis), indépendante du réseau WiFi exposé à l'extérieur.



FIGURE 2 – Robot CIOT TY1251D-03195 "CYBEL"

Architecture générale du robot

Vue dans son ensemble, l'architecture du système peut se lire en quatre couches superposées, illustrées en figure 3 : une couche de présentation et d'interaction, une couche applicative et API, une couche de communication et de services réseau, et une couche matérielle.

Couche de présentation et d'interaction (couche supérieure) : Au sommet de l'architecture se trouve la couche de présentation, qui regroupe l'ensemble des interfaces utilisateur permettant d'interagir avec le robot. On y distingue trois composants principaux : l'**application propriétaire Android**, considérée comme une boîte noire non documentée et hors du périmètre du projet ; l'**interface opérateur web (CYBEL)**, développée dans le cadre du projet et permettant le contrôle et la supervision du robot ; et l'**interface visiteur (mode kiosque)**, dédiée aux interactions simplifiées dans un contexte public. Cette couche constitue le point d'entrée principal des interactions homme-machine et repose sur des échanges réseau vers les couches inférieures, notamment via HTTP et WebSocket. Deux composants complémentaires y sont également présents côté Android : un module de

synthèse vocale (TTS natif), permettant les retours audio du robot, et un mode **WebView kiosque**, utilisé pour des interactions simplifiées embarquées.

Couche applicative et API (couche intermédiaire) : La deuxième couche correspond à la logique applicative centrale du système, matérialisée par l'**API CYBEL développée en FastAPI**. Cette API joue un rôle d'orchestrateur entre les interfaces utilisateur et les mécanismes de communication avec le robot : elle expose des endpoints REST ainsi que des canaux WebSocket pour la supervision en temps réel. Elle est alimentée par deux types de clients un **SDK Python en mode réel**, qui assure la communication directe avec le robot, et un **SDK Python en mode simulé (*mock*)**, permettant le développement et les tests indépendamment du matériel. Ces deux modes convergent vers une interface commune abstraite (**RobotBackend**), garantissant la continuité fonctionnelle entre simulation et exécution réelle.

Couche de communication et de services réseau : La troisième couche représente les interfaces réseau exposées par le robot et utilisées comme points d'accès aux fonctionnalités internes. Elle comprend plusieurs services distincts : **rosbridge** (WebSocket, port 9090), permettant l'envoi et la réception de messages ROS au format JSON ; **rosapi**, utilisé pour l'introspection des topics, services et abonnés ROS ; **MQTT** (port 1883), employé pour la communication événementielle et la télémétrie ; et **ADB** (*Android Debug Bridge*), offrant un accès direct au sous-système Android. Cette couche constitue le point d'interconnexion critique entre la logique applicative CYBEL et le système robotique réel.

Couche matérielle et système robotique : À la base de l'architecture se trouve la couche matérielle, composée des deux sous-systèmes principaux du robot. D'une part, le **châssis de navigation** sous Linux embarqué avec ROS assure les fonctions de mobilité : il intègre les modules de SLAM, de planification de trajectoire (**move_base**) et les capteurs de perception tels que le LiDAR, et reste accessible via l'adresse réseau interne 10.42.0.1. D'autre part, le **module supérieur Android** constitue la partie interactive du robot : il regroupe l'écran tactile, les caméras, les haut-parleurs et les applications utilisateur, et reste accessible via un réseau interne distinct (172.16.0.0/16). Ces deux sous-systèmes communiquent entre eux via la liaison Ethernet interne décrite plus haut, assurant la synchronisation entre navigation et interaction utilisateur.

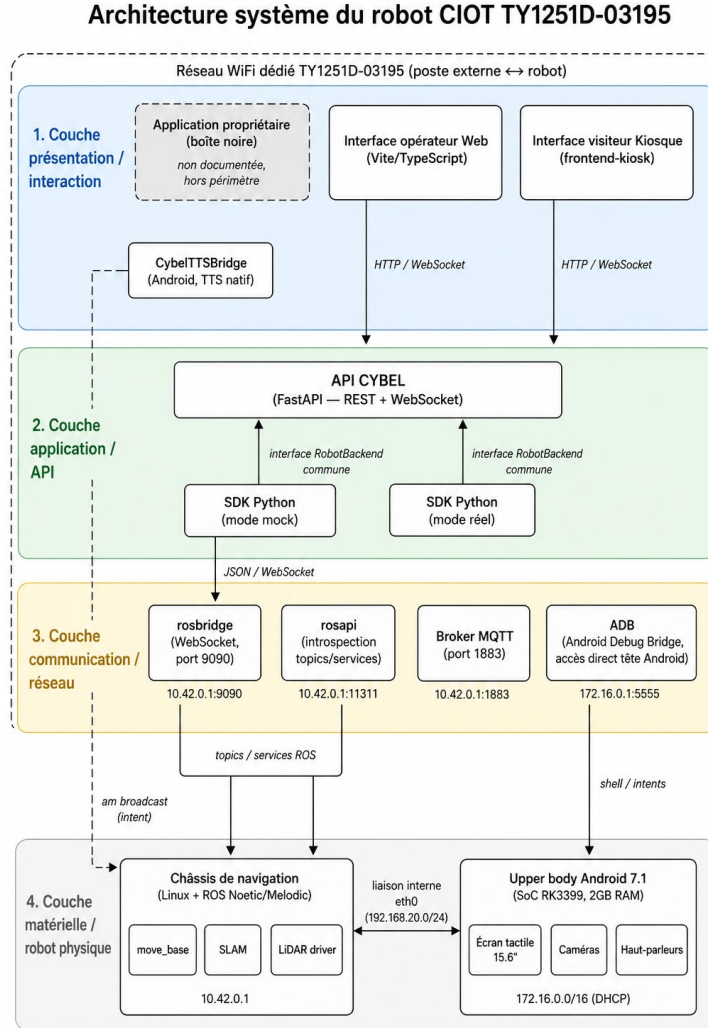


FIGURE 3 – Architecture système globale du robot CIOT TY1251D-03195

Positionnement du système CYBEL dans l’architecture. Le système CYBEL s’insère principalement dans les couches supérieures de l’architecture, en remplaçant et en étendant les capacités de l’application propriétaire. Il s’appuie sur la couche applicative (API FastAPI et SDK Python) pour structurer la logique de commande, sur la couche réseau (rosbridge, MQTT) pour l’interaction avec le robot, et sur les couches matérielles sous-jacentes pour l’exécution des actions physiques. Cette approche permet de découpler totalement l’interface utilisateur du système robotique, tout en garantissant une compatibilité avec l’infrastructure existante.

L’architecture globale du système met ainsi en évidence une organisa-

tion en couches clairement séparées, allant de l'interface utilisateur jusqu'au matériel robotique. Cette structuration constitue le fondement de la solution CYBEL, qui s'inscrit comme une couche logicielle intermédiaire visant à unifier, simplifier et étendre l'accès aux fonctionnalités du robot CIOT TY1251D-03195.

Cadre et déroulement du stage

L'établissement disposant de ce robot souhaite l'utiliser pour des scénarios d'accueil et de démonstration personnalisés : annonces vocales spécifiques, déplacements vers des points définis par l'utilisateur, supervision à distance. Le projet est encadré par HESTIM Engineering & Business School et s'adresse à des étudiants de 3^{ème} ou 4^{ème} année ayant des bases en réseaux, programmation et robotique. Il est prévu sur une durée de trois à quatre mois, structuré en quatre phases : (1) compréhension de l'architecture et établissement de la connectivité réseau, (2) exploration et analyse du protocole de communication interne, (3) développement de l'interface de commande et des modules d'interaction, et (4) intégration, tests et validation finale.

Limites de la solution existante

L'application propriétaire installée sur l'*upper body* ne permet pas le type de personnalisation recherché : elle constitue une boîte noire, sans documentation, sans API exposée et sans code source disponible. Le constructeur ne fournit par ailleurs aucune documentation technique sur les protocoles de communication internes, ce qui interdit toute extension ou adaptation directe de la solution d'origine.

2.2 Problématique

La problématique centrale du projet peut être formulée ainsi :

Comment concevoir une plateforme de commande et d'interaction tierce, fonctionnellement équivalente ou supérieure à l'application propriétaire d'un robot de service Android, en l'absence de toute documentation officielle sur ses interfaces de communication internes ?

Cette problématique se décline en plusieurs sous-questions :

- Quels sont les **points d'accès réseau** (adresses IP, ports, protocoles) exposés par le robot, et lesquels sont effectivement exploitables sans authentification propriétaire ?

- Quel est le **format des messages** échangés entre l’interface Android et le système de navigation (ROS), et peut-il être reconstruit par observation et introspection plutôt que par décompilation de l’application ?
- Comment **distinguer une commande effectivement traitée par le robot d’une commande silencieusement ignorée**, dans un système où le protocole de transport (`rosbridge`) ne renvoie pas systématiquement d’erreur explicite ?
- Comment concevoir une architecture logicielle qui reste **utilisable et testable sans accès permanent au robot physique** (contraintes de disponibilité du matériel), tout en restant fidèle au comportement réel une fois connectée ?
- Dans quelle mesure les fonctionnalités d’**interaction homme-robot** (synthèse vocale, reconnaissance vocale, retour visuel) peuvent-elles être reconstruites lorsque le canal correspondant n’est pas exposé par le système de navigation (ROS) et nécessite un accès direct au sous-système Android ?

2.3 Objectifs du projet

2.3.1 Objectif général

Concevoir et développer une **plateforme web indépendante** de commande, de supervision et d’interaction pour le robot CIOT TY1251D-03195, sans dépendre de l’application Android propriétaire, en documentant de façon rigoureuse le **protocole de communication** reconstruit (endpoints, topics, services, formats de messages) afin que ce travail puisse être réutilisé ou étendu.

2.3.2 Objectifs spécifiques

1. **Établir la connectivité** avec le robot (réseau WiFi dédié, identification des hôtes et ports actifs).
2. **Identifier les points de communication** exposés : WebSocket `rosbridge` (port 9090), broker MQTT (port 1883), services FTP/SSH, interfaces web internes (:8082, :8088).
3. **Capturer et analyser le trafic réseau** afin de reconstituer les topics et services ROS pertinents (mouvement, navigation, télémétrie, capteurs).
4. **Décoder et reconstruire les commandes de contrôle** du mouvement et de la navigation (vitesse, position cible, annulation de

trajectoire).

5. **Développer une interface de commande web** (backend FastAPI + frontend TypeScript) permettant la supervision temps réel et l'envoi de commandes.
6. **Implémenter des fonctionnalités d'interaction homme-robot de base** : navigation par sélection de points ou clic sur carte, retour vocal (TTS) et commande vocale opérateur.
7. **Mettre en place un mode de simulation** (« mock ») permettant de développer et tester l'interface sans dépendre de la disponibilité physique du robot.
8. **Intégrer l'ensemble des composants** (SDK Python, API, interface web) dans un système cohérent et démontrable.

2.4 Cahier des charges

2.4.1 Périmètre du projet

Le tableau 1 délimite les activités prises en charge par le projet de celles volontairement laissées hors périmètre.

Inclus dans le périmètre	Exclu du périmètre
Identification et documentation des canaux de communication du robot (rosbridge, MQTT, services réseau)	Modification du firmware ou du logiciel embarqué du robot
Développement d'un SDK Python d'accès au robot (mode simulé et mode réel)	Développement d'une application Android de remplacement (optionnel, non engagé)
Développement d'une API de commande/supervision (FastAPI)	Décompilation ou rétro-ingénierie de l'application Android propriétaire
Développement d'une interface web opérateur (Vite/TypeScript)	Déploiement en production / hébergement distant
Fonctionnalités de navigation (point nommé, clic sur carte, téléopération)	Comportements autonomes avancés (planification de mission, multi-robot)
Interaction de base : synthèse vocale (TTS) et commande vocale opérateur	Vision par caméra, reconnaissance faciale, chatbot (extensions optionnelles)
Documentation technique et rapport de stage	Formation des utilisateurs finaux / exploitation au long cours

TABLE 1 – Périmètre du projet : éléments inclus et exclus

2.4.2 Acteurs du système

Le tableau 2 identifie les principaux acteurs impliqués dans le projet ainsi que leur rôle respectif.

Acteur	Rôle
Étudiant(e) stagiaire	Conception, développement, rétro-ingénierie, rédaction du rapport
Encadrant académique (Dr. Sridath Tula, HESTIM)	Suivi pédagogique, validation des objectifs et du rapport
Établissement d'accueil	Mise à disposition du robot et de l'environnement de travail
Opérateur final	Supervision et pilotage du robot via la plateforme CYBEL
Robot CIOT TY1251D-03195	Système cible — serveur de télémétrie et de commandes via <code>rosbridge</code> /MQTT

TABLE 2 – Acteurs du système et leurs rôles

2.4.3 Besoins fonctionnels

Le tableau 8 liste les exigences fonctionnelles du système, accompagnées de leur niveau de priorité.

Réf.	Exigence	Priorité
EF-01	Superviser en temps réel l'état du robot (batterie, position, statut, mode de navigation)	Essentielle
EF-02	Afficher la carte SLAM, le LiDAR et les personnes détectées	Essentielle
EF-03	Naviguer vers un point nommé prédéfini	Essentielle
EF-04	Naviguer vers une coordonnée choisie par clic sur la carte, avec prise en compte des obstacles	Essentielle
EF-05	Téléopérer manuellement le robot (vitesse linéaire/angular) avec arrêt d'urgence	Essentielle
EF-06	Déclencher une annonce vocale sur le robot (TTS)	Souhaitable
EF-07	Donner des commandes vocales à l'opérateur via le micro du navigateur	Souhaitable
EF-08	Configurer les paramètres de fonctionnement (vitesse, mode de déplacement)	Souhaitable
EF-09	Fonctionner en mode simulation complet sans robot connecté	Essentielle

TABLE 3 – Besoins fonctionnels du système

2.4.4 Besoins non fonctionnels

Le tableau 9 présente les exigences non fonctionnelles, relatives notamment à la performance, à la robustesse et à la séparation des responsabilités au sein de l'architecture.

Réf.	Exigence	Priorité
ENF-01	Latence d'affichage de la télémétrie de l'ordre de quelques centaines de ms maximum	Essentielle
ENF-02	Reconnexion automatique au robot après perte de connexion réseau	Essentielle
ENF-03	Séparation stricte entre couche d'accès robot (SDK), API et présentation	Essentielle
ENF-04	Toute commande de mouvement/navigation doit être annulable à tout moment	Essentielle
ENF-05	Aucune action exploratoire ne doit perturber le fonctionnement normal du robot	Essentielle

TABLE 4 – Besoins non fonctionnels du système

2.4.5 Contraintes du projet

- **Contraintes techniques** : absence de documentation officielle du protocole ; dépendance à un réseau WiFi dédié et potentiellement instable (latence variable, de 7 à 1654 ms observés) ; version Android embarquée obsolète (7.1) limitant les outils de débogage disponibles.
- **Contraintes matérielles** : un seul exemplaire de robot disponible, partagé avec d'autres usages de l'établissement — disponibilité limitée pour les tests.
- **Contraintes organisationnelles** : durée de projet limitée à trois-quatre mois ; étudiant unique sur le projet ; encadrement académique périodique.
- **Contraintes de sécurité et d'éthique** : les canaux `rosbridge` et MQTT du robot étant accessibles sans authentification, toute manipulation doit respecter un principe de **précaution** (vérification en mode simulation avant toute commande réelle, mécanisme d'annulation disponible) ; les tentatives d'accès aux comptes système (SSH/FTP) doivent rester mesurées pour ne pas provoquer de blocage.
- **Contraintes de propriété intellectuelle** : aucune décompilation du logiciel propriétaire n'est entreprise ; seule l'observation du trafic réseau et l'introspection des interfaces standard (ROS, MQTT) sont utilisées.

2.4.6 Livrables attendus

Conformément au sujet de stage, les livrables attendus en fin de projet sont :

1. Une **interface de communication fonctionnelle** avec le robot (SDK + client `rosbridge`/MQTT).
2. Un **système de contrôle du mouvement** opérationnel (téléopération + navigation).
3. Une **interface utilisateur** web opérateur.
4. Un **module d'interaction de base** (tactile et/ou vocal).
5. Un **rapport technique** incluant l'architecture du système et l'analyse du protocole.

2.4.7 Critères de réception et de validation

Le tableau 5 précise, pour chaque critère d'acceptation, la modalité de vérification associée.

Critère	Modalité de vérification
Connexion au robot établie de façon reproductible	Connexion <code>rosbridge</code> réussie depuis le réseau WiFi du robot, vérifiable via <code>/rosapi/topics</code>
Télémétrie affichée en temps réel	Observation du tableau de bord (position, batterie, statut) à jour
Navigation vers un point ou une coordonnée	Envoi d'une commande de navigation suivie d'un changement d'état (<code>/navi_status</code>) cohérent
Téléopération et arrêt d'urgence	Test en mode simulation puis, si possible, sur robot réel à vitesse réduite
Fonctionnement en mode simulation	Lancement avec <code>ROBOT MOCK=true</code> , toutes les fonctionnalités principales accessibles
Interaction vocale	TTS robot ou solution de repli (voix opérateur via navigateur) fonctionnelle
Documentation	Présence et cohérence de <code>README.md</code> , <code>docs/INTERFACE.md</code> et du présent rapport

TABLE 5 – Critères de réception et de validation

2.5 Solution proposée

Face à ce besoin, la solution retenue consiste en une plateforme logicielle indépendante, structurée autour d'une couche d'accès au robot (SDK Python interchangeable mock/réel), d'une API de commande et de supervision, et d'interfaces utilisateur web dédiées à l'opérateur et au visiteur. L'analyse fonctionnelle détaillée, l'architecture logicielle complète et les diagrammes de conception associés sont présentés au Chapitre 4 (*Analyse et conception du système*).

2.6 Conclusion

Ce chapitre a permis de poser le cadre du projet CYBEL : l'architecture du robot CIOT TY1251D-03195 et les limites de sa solution propriétaire, la problématique de rétro-ingénierie et de conception qui en découle, les objectifs poursuivis, ainsi qu'un cahier des charges détaillant acteurs, besoins, contraintes, livrables et critères de validation. Le chapitre suivant situe ce travail par rapport à l'état de l'art en robotique de service, architectures logicielles robotiques et interaction homme-robot.

3 État de l’art

Ce chapitre situe le projet CYBEL par rapport aux travaux et solutions existants. Il présente les fondements théoriques de la robotique de service ainsi que les solutions logicielles et matérielles comparables, afin de mettre en évidence les limites des approches actuelles et de justifier les choix architecturaux retenus.

3.1 Fondements théoriques

La conception de la plateforme s’appuie sur plusieurs corpus de référence :

- **Architecture logicielle robotique** : le modèle ROS (topics, services, actions) proposé par Quigley et al. constitue le paradigme de communication analysé dans ce projet.
- **Navigation autonome** : les concepts de cartes d’occupation et de planification (costmaps, DWA) décrits dans *Probabilistic Robotics* (Thrun, Burgard & Fox, 2005) servent de base à l’interprétation du module `move_base`.
- **Protocole rosbridge** : le standard JSON de communication ROS (subscribe, publish, call_service) est utilisé comme référence pour la reconstruction des échanges observés.
- **Interaction homme-robot (HRI)** : les principes de supervision en temps réel et de sécurité opérationnelle (priorité à l’arrêt d’urgence) guident la conception de l’interface CYBEL.
- **Sécurité IoT** : les recommandations de l’OWASP IoT Top 10 permettent d’analyser les risques liés aux canaux non authentifiés observés sur le système.

3.2 État des solutions existantes

Les solutions suivantes représentent les principales approches disponibles dans l’écosystème robotique et du robot étudié.

3.3 Analyse comparative

3.4 Limites des solutions existantes

Les solutions étudiées présentent plusieurs limites majeures :

- Absence d’extensibilité des systèmes propriétaires
- Outils ROS de trop bas niveau pour des usages métier

Solution	Description	Accessibilité
Application propriétaire (upper body Android)	Interface tactile constructeur pour accueil et navigation	Fermée, sans API ni documentation
Interface web embarquée (:8082)	Outil de gestion des cartes SLAM	Accessible localement, non documenté
Interface de debug (:8088)	Interface interne de diagnostic	Non documentée, usage inconnu
rosbridge / RViz / Foxglove	Outils ROS de visualisation et contrôle	Open-source mais génériques
SDK robots commerciaux (Temi, Pepper, etc.)	SDK propriétaires pour robots de service	Documentés mais verrouillés

TABLE 6 – Panorama des solutions existantes comparables

Critère	App. propriétaire	RViz / Foxglove	CYBEL
Documentation	Aucune	Open-source	Documenté (rapport + code)
Accès distant web	Non	Partiel	Oui
Personnalisation scénarios	Non	Non	Oui
Mode simulation	Non	Partiel	Oui (mock)
Navigation interactive	Oui	Oui	Oui + logique métier
Interaction vocale	Oui	Non	Partiel / évolutif

TABLE 7 – Benchmark comparatif des solutions

- Aucune solution ne propose de mode déconnecté du robot
- Documentation constructeur insuffisante nécessitant rétro-ingénierie

3.5 Synthèse

L’analyse met en évidence un manque de solutions adaptées à un usage orienté **scénarios d’accueil personnalisés** et **supervision haut niveau**. Ces limites justifient le développement de la plateforme CYBEL, détaillée

dans le chapitre suivant.

3.6 Conclusion

Ce chapitre a permis de situer le projet CYBEL par rapport aux fondements théoriques de la robotique de service architecture ROS, navigation autonome, protocole `rosbridge`, interaction homme-robot et sécurité des objets connectés ainsi qu’aux solutions existantes, qu’elles soient propriétaires (application Android constructeur) ou génériques (`rosbridge`, `RViz`, `Foxglove`, SDK de robots commerciaux). L’analyse comparative a montré qu’aucune de ces solutions ne répond pleinement aux besoins identifiés au Chapitre 2 : extensibilité, logique métier orientée accueil, supervision de haut niveau et mode de développement déconnecté du robot physique. Ces limites confirment la pertinence du projet CYBEL et orientent directement les choix de conception présentés au Chapitre 4, qui détaille l’analyse fonctionnelle et l’architecture logicielle de la solution retenue.

4 Analyse et conception du système

Ce chapitre présente l'analyse fonctionnelle du système CYBEL ainsi que les choix de conception retenus pour répondre au cahier des charges défini au chapitre 2, à la lumière de l'analyse critique des solutions existantes présentée au chapitre 3. Il détaille successivement les acteurs et les besoins identifiés, l'architecture générale et logicielle de la plateforme, les diagrammes de conception (composants, cas d'utilisation, séquence) et le modèle des flux de communication entre les différents sous-systèmes.

4.1 Analyse fonctionnelle

Après l'étude du besoin et l'analyse des solutions existantes, il est nécessaire de définir précisément le fonctionnement du système à concevoir. Cette phase d'analyse fonctionnelle permet d'identifier les acteurs intervenant dans le système, les services qui leur sont proposés ainsi que les interactions nécessaires à la réalisation des différentes fonctionnalités.

Le projet CYBEL vise à fournir une plateforme de supervision, de commande et d'interaction destinée au robot de service CIOT TY1251D-03195, déployé au sein du FabLab de HESTIM. Cette plateforme doit permettre aux opérateurs de piloter le robot, de suivre son état en temps réel et de gérer les interactions avec les visiteurs, tout en assurant une communication fiable avec les différents composants matériels et logiciels du système.

L'analyse fonctionnelle constitue ainsi une étape essentielle avant la conception détaillée de l'architecture, puisqu'elle permet de traduire les besoins exprimés dans le cahier des charges (section 8 et suivantes) en fonctionnalités concrètes, et de les rattacher explicitement aux exigences EF/ENF formulées au chapitre 2.

4.1.1 Identification des acteurs

L'étude des besoins a permis d'identifier trois acteurs principaux intervenant dans le fonctionnement du système CYBEL.

L'opérateur : L'opérateur représente l'utilisateur chargé de l'administration et de la supervision du robot. Il dispose d'une interface lui permettant de consulter l'état du robot, de visualiser sa position sur la carte, d'accéder aux informations de télémétrie, de lancer des scénarios de navigation, de téléopérer le robot, de déclencher des annonces vocales et de gérer les paramètres du système. Son rôle est central puisque c'est lui qui assure le contrôle opérationnel du robot au sein du FabLab.

Le visiteur : Le visiteur interagit avec le robot à travers l'interface embarquée sur la tablette Android. Cette interface lui permet notamment de découvrir les différents espaces du FabLab, de sélectionner des points d'intérêt, de consulter des informations de présentation et d'interagir avec le robot lors des démonstrations. Le visiteur n'a pas accès aux fonctionnalités d'administration ou de contrôle du système.

Le système CYBEL : Le système CYBEL constitue un acteur autonome chargé de coordonner les différents composants logiciels. Il assure notamment la collecte des données du robot, la diffusion de la télémétrie, la gestion des communications, la reconnexion automatique en cas de perte de liaison, la synchronisation des données entre les interfaces, ainsi que le déclenchement des services associés à la navigation et à l'interaction.

4.1.2 Besoins fonctionnels

L'analyse des besoins réalisée au cours des phases préliminaires du projet a permis d'identifier les principales fonctionnalités que le système CYBEL doit assurer afin de répondre aux attentes des utilisateurs et aux contraintes du robot. Les besoins fonctionnels retenus, repris du cahier des charges établi au chapitre 2 (réf. EF-01 à EF-09), sont présentés dans le tableau 8 avec un rajout de quelque fonctionnalité identifier par la suite (EF-10 à EF-11).

TABLE 8 – Besoins fonctionnels du système CYBEL, synthèse par fonctionnalité

Réf.	Fonctionnalité	Description
EF-01, EF-02	Supervision du robot	Consultation en temps réel du niveau de batterie, de la position du robot, de son état de navigation, de la carte SLAM, des données LiDAR, des visiteurs détectés et des messages de diagnostic.
EF-03, EF-04	Navigation autonome	Envoi du robot vers un point prédéfini, navigation vers une position sélectionnée sur la carte, suivi de mission et annulation d'une tâche en cours.
EF-05	Téléopération	Contrôle manuel des déplacements du robot via une interface de commande, avec arrêt d'urgence.
EF-06, EF-07	Interaction Homme-Robot	Déclenchement d'annonces vocales (TTS), diffusion de messages d'accueil et commandes vocales données par l'opérateur.
EF-08	Configuration du système	Modification des paramètres de fonctionnement tels que les vitesses de déplacement, les paramètres de navigation et les réglages de communication.
EF-09	Mode simulation	Exécution du système sans robot physique afin de faciliter les phases de développement et de test.
EF-10	Interface visiteur kiosque	Accès tactile dédié, indépendant de l'interface opérateur, proposant des actions d'accueil et une foire aux questions bilingue (FR/EN) destinées au visiteur.
EF-11	Déploiement autonome	Exécution de la plateforme directement sur la tablette du robot, sans dépendance à un poste de développement externe.

4.1.3 Besoins non fonctionnels

Au-delà des fonctionnalités attendues, le système doit respecter un ensemble d'exigences de qualité garantissant sa robustesse, sa fiabilité et son utilisabilité. Les besoins non fonctionnels identifiés, repris du chapitre 2 (réf. ENF-01 à ENF-05), sont présentés dans le tableau 9.

TABLE 9 – Besoins non fonctionnels du système CYBEL, synthèse par critère

Réf.	Critère	Description
ENF-01	Performance	Mise à jour rapide des informations (télémétrie de l'ordre de quelques centaines de millisecondes maximum) afin d'assurer une supervision efficace et une interaction fluide avec le robot.
ENF-02	Fiabilité	Maintien de la continuité des communications et reconnexion automatique en cas d'interruption réseau.
ENF-03	Maintenabilité	Architecture modulaire, séparant strictement la couche d'accès au robot, l'API et la présentation, facilitant l'évolution et la maintenance du système.
ENF-04	Sécurité opérationnelle	Possibilité d'annuler à tout moment toute commande de mouvement ou de navigation en cours.
ENF-05	Innocuité des phases exploratoires	Les scripts utilisés lors de la rétro-ingénierie du protocole ne doivent en aucun cas perturber le fonctionnement normal du robot.
	Ergonomie	Interfaces intuitives et adaptées aussi bien aux opérateurs qu'aux visiteurs.

4.2 Architecture générale de CYBEL

Afin de répondre aux besoins identifiés, une architecture modulaire a été retenue. Celle-ci repose sur plusieurs composants indépendants communiquant entre eux à travers des interfaces clairement définies. L'objectif principal de cette architecture est de découpler les interfaces utilisateur du système robotique afin de faciliter les tests, la maintenance et l'évolution de la plateforme.

Comme illustré en figure 4, l'architecture globale du système s'articule, d'un point de vue physique et déploiement, autour de quatre ensembles principaux : le robot CIOT TY1251D-03195, le backend CYBEL, les interfaces utilisateur et les applications Android embarquées. La sous-section suivante propose une seconde vue de ce même système, cette fois d'un point de vue logique, organisée en couches logicielles.

4.3 Architecture logicielle

L'architecture logicielle retenue suit une approche en couches permettant de séparer les responsabilités fonctionnelles et techniques du système. Elle est organisée autour de trois couches principales, présentées en figure 5.

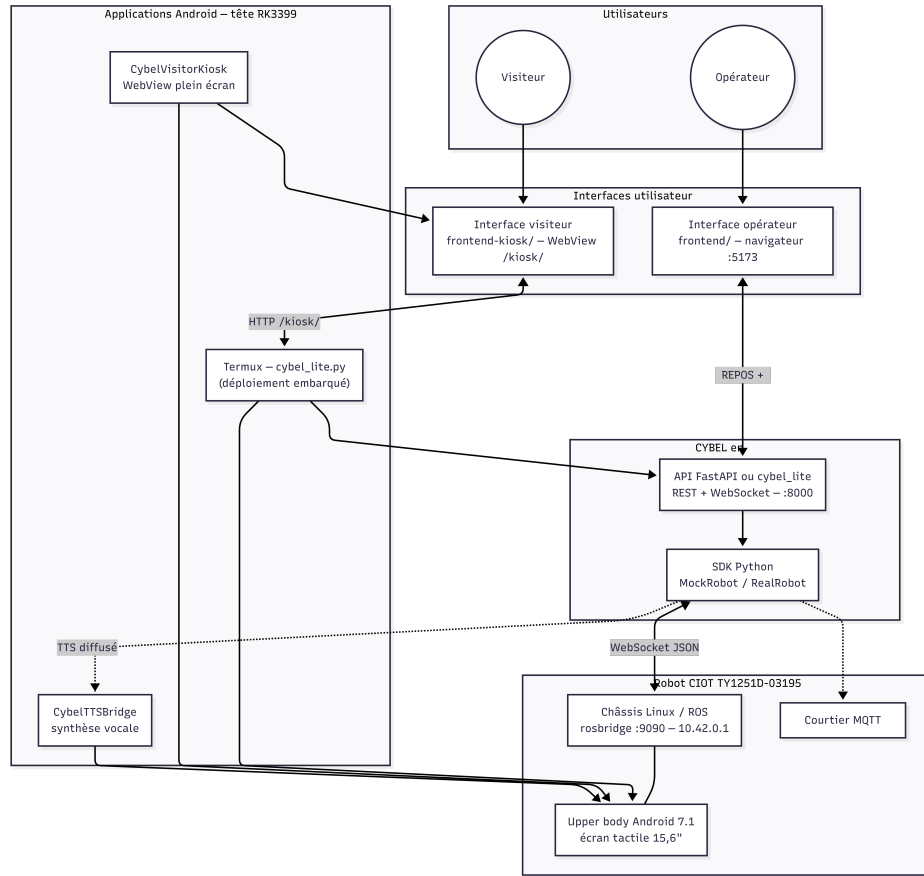


FIGURE 4 – Architecture générale du système CYBEL

4.3.1 Couche d'accès au robot (SDK)

Cette couche constitue l'abstraction du robot. Elle est responsable de la communication avec *robridge*, de la gestion du protocole reconstruit, de l'accès aux données de télémétrie et de l'exécution des commandes. Deux implémentations sont proposées : une implémentation réelle et une implémentation simulée. Cette approche permet de développer et de tester la plateforme sans dépendre de la disponibilité permanente du robot.

4.3.2 Couche API

Cette couche repose sur FastAPI et constitue le point d'entrée principal du système. Elle expose des services REST, des services WebSocket ainsi que les mécanismes de gestion des événements. Elle joue également le rôle d'intermédiaire entre les interfaces utilisateur et le SDK.

4.3.3 Couche présentation

Cette couche regroupe l'interface opérateur, l'interface visiteur et les applications Android complémentaires. Elle permet l'interaction entre les utilisateurs et le système.

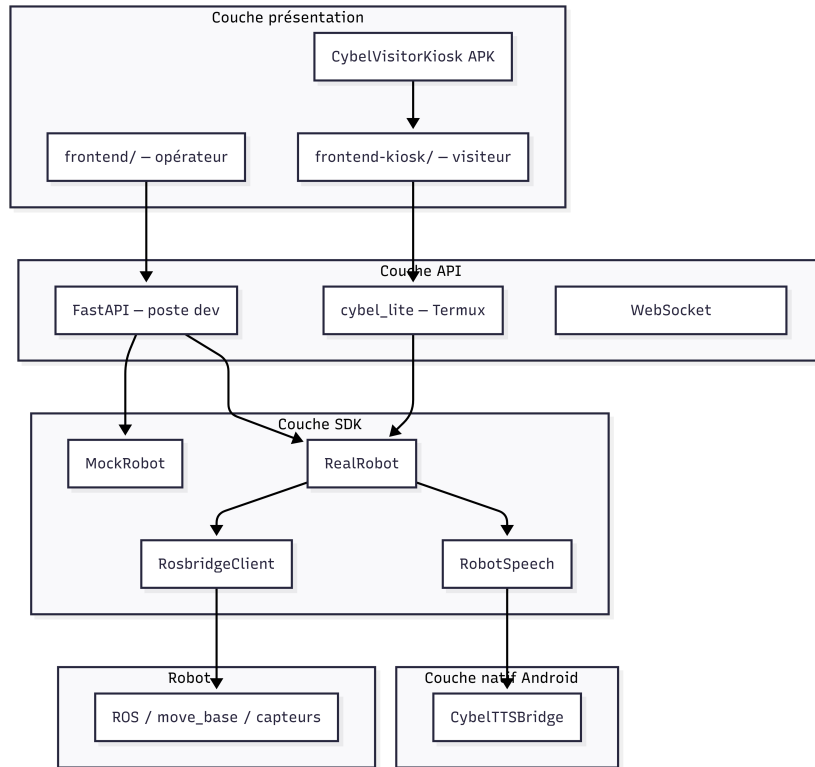


FIGURE 5 – Architecture en couches du système CYBEL

4.4 Diagramme des composants

Le diagramme des composants (figure 6) met en évidence l'organisation des différents modules logiciels du système ainsi que leurs dépendances. Il montre notamment les interfaces opérateur et visiteur, le backend FastAPI, le SDK robot, *rosbridge*, le système Android embarqué ainsi que les modules TTS. Cette représentation permet de visualiser clairement la répartition des responsabilités entre les différents composants.

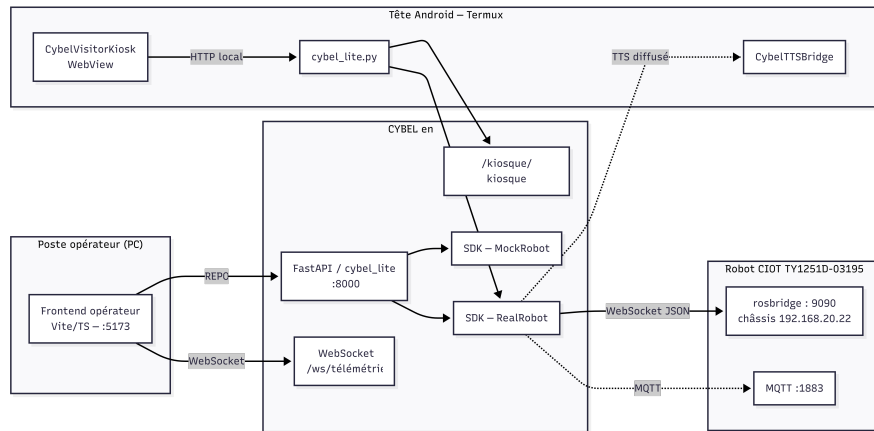


FIGURE 6 – Diagramme de composants du système CYBEL

4.5 Diagrammes de cas d'utilisation

Les diagrammes de cas d'utilisation (figure 7) permettent de représenter les interactions entre les acteurs identifiés et les fonctionnalités proposées par le système. Ils mettent en évidence les actions réalisées par l'opérateur, les interactions du visiteur, ainsi que les traitements automatiques assurés par CYBEL.

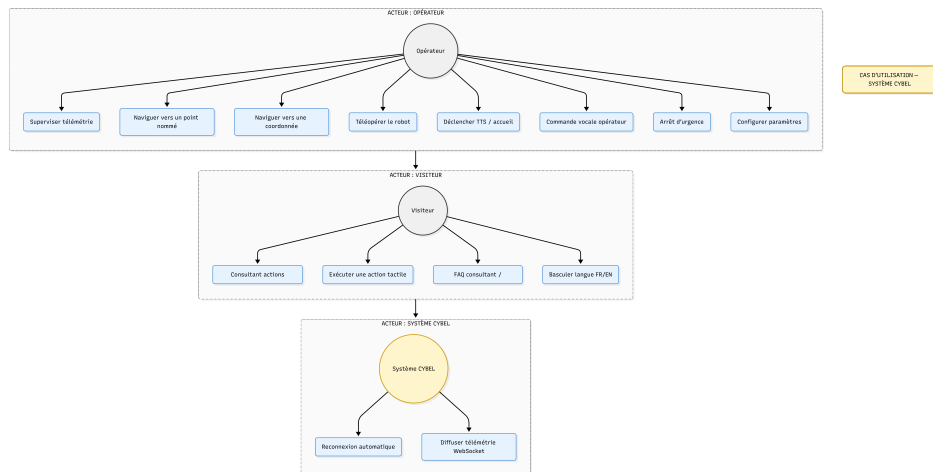


FIGURE 7 – Diagramme de cas d'utilisation du système CYBEL

4.6 Diagrammes de séquence

Les diagrammes de séquence (figure 8) permettent de décrire l'enchaînement des échanges entre les différents composants du système lors de l'exécution d'une fonctionnalité. Parmi les scénarios étudiés figurent la navigation vers un point sélectionné, l'envoi d'une commande de déplacement, le déclenchement d'une annonce vocale, ainsi que la diffusion de la télémétrie vers l'interface opérateur. Ces diagrammes facilitent la compréhension des mécanismes internes mis en œuvre lors des communications entre le robot, le backend et les interfaces utilisateur.

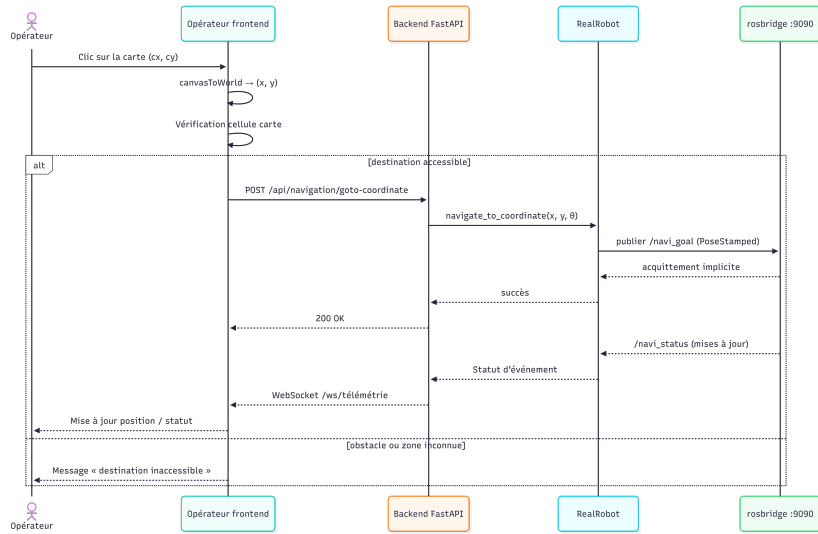


FIGURE 8 – Diagramme de séquence de la navigation vers un point sélectionné

4.7 Modèle des flux de communication

4.7.1 Robot ↔ rosbridge

rosbridge constitue la passerelle principale entre la plateforme CYBEL et le système de navigation du robot. Les échanges reposent sur le protocole WebSocket et permettent la réception des données de télémétrie, l'abonnement aux topics ROS, l'appel de services, ainsi que l'envoi de commandes de navigation et de mouvement.

4.7.2 **rosbridge ↔ Backend**

Le backend agit comme intermédiaire entre *rosbridge* et les interfaces utilisateur. Il collecte les informations provenant du robot, les traite puis les redistribue vers les clients connectés. Il assure également la gestion des commandes envoyées par les utilisateurs.

4.7.3 **Backend ↔ Interfaces utilisateur**

Les interfaces communiquent avec le backend à travers des requêtes REST pour les actions ponctuelles et des WebSockets pour les mises à jour temps réel. Cette architecture garantit une supervision réactive et une faible latence lors de l’affichage des données.

4.7.4 **Backend ↔ Applications Android**

Les applications Android développées dans le cadre du projet permettent d’étendre les fonctionnalités du système, notamment pour la synthèse vocale, l’affichage du mode kiosque et les interactions avec la tablette embarquée.

4.8 **Conclusion**

Ce chapitre a permis de présenter l’analyse fonctionnelle et la conception générale du système CYBEL. Les besoins identifiés, repris et détaillés à partir du cahier des charges du chapitre 2, ont conduit à la définition d’une architecture modulaire organisée autour d’un SDK robot, d’une API centralisée et de plusieurs interfaces utilisateur. Les différents diagrammes présentés permettent de visualiser les interactions entre les acteurs et les composants du système ainsi que les flux de communication mis en œuvre. Cette phase de conception constitue la base de la démarche méthodologique présentée au chapitre 5, puis des développements réalisés au chapitre 6, consacré à la réalisation et à l’implémentation de la solution proposée.

5 Méthodologie et planification

La planification opérationnelle est un processus essentiel pour structurer et coordonner l'exécution des tâches dans une organisation ou un projet. Elle repose sur des méthodes rigoureuses, des applications pratiques et des outils spécialisés pour assurer une gestion efficace des ressources, des délais et des objectifs.

Ce glossaire définit les principaux concepts liés à la planification opérationnelle et explore les méthodes, applications et outils qui permettent d'optimiser son exécution.

Dans le cadre de notre stage, nous avons appliqué une démarche méthodique et structuré notre travail pour atteindre les objectifs fixés. Ce chapitre détaille notre mode de travail, la planification, les outils employés ainsi que les étapes concrètes de mise en œuvre.

5.1 Mode de travail et organisation

Dans le besoin d'un travail optimale, le mode de travail qui s'est avéré étant la plus optimale pour notre stage fut la répartition des tâches en optant sur nos points fort. Cette méthode nous auras permis de renforcer nos acquis sans pour autant rester dans notre zone de confort car malgré cette méthode de travail, il y a eu derrière un grand travail d'autonomie pour approfondir la documentation ou le développement des scripts Python présent au sein du compte GitHub **Claude20022002** sur lesquelles nous devons travailler. Notre superviseur ainsi que tous les membres du laboratoire nous ont supervisés à travers des réunions, compte rendus journalier et hebdomadaire. Pour organiser notre travail, nous avons utilisé un planning détaillé et avons tenu des comptes rendus partagés sur Github où l'on présentait nos problèmes, hypothèses et solutions.

5.2 Planning du projet

Afin d'assurer une progression cohérente tout au long du stage et de respecter les objectifs fixés, un planning prévisionnel a été établi dès le début du projet sous la forme d'un diagramme de Gantt. Celui-ci a permis de répartir les différentes activités sur toute la durée du stage, d'identifier les dépendances entre les tâches et de suivre l'avancement du projet.

Le déroulement du projet a été organisé en quatre grandes phases successives.

La première phase, consacrée à la **connectivité**, avait pour objectif de comprendre l'architecture matérielle et réseau du robot. Cette étape a

consisté à analyser la topologie du système, identifier les différents équipements présents sur le réseau et réaliser un balayage des ports afin de découvrir les services accessibles.

La deuxième phase concernait l'**investigation du système**. Durant cette étape, plusieurs travaux de rétro-ingénierie ont été réalisés afin d'identifier les mécanismes de communication utilisés par le robot. Les services ROS ont été explorés grâce à `rosapi`, les échanges MQTT ont été analysés afin de comprendre les commandes disponibles, tandis que le fonctionnement du système de synthèse vocale a été étudié à l'aide d'ADB et d'un pont de communication Android.

La troisième phase correspond au **développement de la plateforme CYBEL**. Cette étape a constitué la partie la plus importante du projet. Elle a conduit au développement d'un SDK Python permettant de communiquer avec le robot en mode simulé ou réel, à la réalisation du backend avec FastAPI et WebSocket, à la conception de l'interface web destinée aux opérateurs ainsi qu'au développement du moteur de visite guidée, du kiosque interactif destiné aux visiteurs et de la solution de déploiement sur la tablette Android via Termux.

Enfin, la quatrième phase est dédiée à la **validation du système**. Elle comprend les essais de navigation sur le robot réel, l'ajustement des coordonnées des différents points du parcours de visite, les tests d'intégration de l'ensemble des modules développés ainsi que la rédaction du rapport de stage et la préparation de la soutenance.

La Figure 9 présente le planning global du projet et l'enchaînement des différentes phases de réalisation.

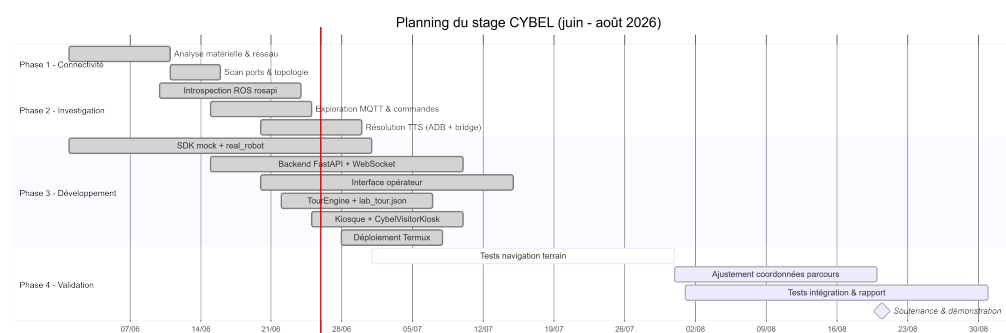


FIGURE 9 – Diagramme de Gantt du projet CYBEL

Comme le montre ce planning, plusieurs activités ont été menées en parallèle, notamment durant la phase de développement. Cette organisation a permis d'exploiter efficacement le temps disponible pendant les périodes où

le robot n'était pas accessible, en poursuivant le développement de l'interface web, du backend ou des modules de simulation avant leur intégration sur le robot réel.

5.3 Outils, Langage et Technologie utilisés

5.3.1 ROS

5.3.2 FastAPI

5.3.3 TypeScript

5.3.4 Android

5.3.5 ADB

5.3.6 ROS

5.4 Découverte du Robot CIOT TY1251D-03195 "CY-BEL"

5.5 Proposition de documentation technique

5.6 Apports du stage

5.7 Difficultés rencontrées durant la phase d'investigation

5.8 Conclusion

6 Réalisation et implémentation

6.1 Mise en place de l'environnement

6.2 Découverte et analyse du robot

6.3 Reverse Engineering du système

6.3.1 Analyse réseau

6.3.2 Identification des protocoles

6.3.3 Étude des commandes

6.4 Développement du backend CYBEL

6.5 Développement de l'application Android

6.6 Développement de l'interface Web

6.7 Mise en place du système TTS

6.8 Intégration des différents modules

6.9 Conclusion

7 Validation, résultats et analyse critique

7.1 Stratégie de validation

7.2 Scénarios de tests

7.3 Résultats obtenus

7.4 Indicateurs de performance

Tableau avec : Temps de réponse Fiabilité Stabilité des communications
Robustesse

7.5 Contributions personnelles

7.6 Difficultés rencontrées et solutions apportées

7.7 Limites du projet

7.8 Perspectives d'amélioration

7.9 Conclusion

CONCLUSION GÉNÉRALE

BIBLIOGRAPHIE

WEBOGRAPHIE

1. Robot Web Tools - rosbridge_suite : https://github.com/RobotWebTools/rosbridge_suite
2. Robot Web Tools - roslibjs : <https://github.com/RobotWebTools/roslibjs>
3. FastAPI - documentation officielle : <https://fastapi.tiangolo.com>
4. Vite - documentation officielle : <https://vitejs.dev>
5. Pydantic - documentation : <https://docs.pydantic.dev>
6. Mozilla Developer Network - Web Speech API : https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
7. Android Developers - TextToSpeech : <https://developer.android.com/reference/android/speech/tts/TextToSpeech>
8. Android Developers - Debug Bridge (ADB) : <https://developer.android.com/tools/adb>
9. Termux Wiki : <https://wiki.termux.com>
10. Mermaid - diagrammes as code : <https://mermaid.js.org>
11. Mermaid Live Editor (export PNG pour Overleaf) : <https://mermaid.live>
12. Claude AI : <https://claude.ai/new>

ANNEXES

Annexe 1 : Capture d'écran



FIGURE 10 – Carte du laboratoire

Annexe 2 : Commandes utilisées

Listing 1 – Commandes de lancement

```
# Verification reseau
ping 10.42.0.1
python scripts/robot_status.py
```

```
PS C:\Users\clusa> ssh -p 8022 u0_a92@172.16.0.132
PS C:\Users\clusa> ssh -p 8022 u0_a92@172.16.0.131
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@    WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!    @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
The fingerprint for the ED25519 key sent by the remote host is
SHA256:2QrvZe29VR/PVLjRtQnhkI3oJL0m0avLtD/FpGff97E.
Please contact your system administrator.
Add correct host key in C:\Users\clusa/.ssh/known_hosts to get rid of
this message.
Offending ED25519 key in C:\Users\clusa/.ssh/known_hosts:10
Host key for [172.16.0.131]:8022 has changed and you have requested strict
checking.
Host key verification failed.
PS C:\Users\clusa> ssh-keygen -R [172.16.0.132]:8022
# Host [172.16.0.132]:8022 found: line 11
# Host [172.16.0.132]:8022 found: line 12
# Host [172.16.0.132]:8022 found: line 13
C:\Users\clusa/.ssh/known_hosts updated.
Original contents retained as C:\Users\clusa/.ssh/known_hosts.old
```

FIGURE 11 – Génération de clé SSH

```
# Lancement developpement
python scripts/dev.py

# Tests
python -m pytest tests/ -v

# TTS test (USB branche)
adb shell am broadcast -n com.cybel.ttsbridge/.
    SpeakReceiver -a com.cybel.ttsbridge.SPEAK --es text "
    Test HESTIM"

# Deploiement tablette
python scripts/deploy_termux.py --skip-kiosk-build --lite
    -only --host <IP> --password <mdp>

# Termux (tablette)
cd ~/cybel/scripts/termux
./stop_cybel.sh && ./start_cybel.sh
curl http://127.0.0.1:8000/api/health
```

Annexe 3 : Code source (extrait)

```
function hello() {
```



```

    console.log("Bonjour !");
}

```

Annexe 4 : Diagrammes détaillés

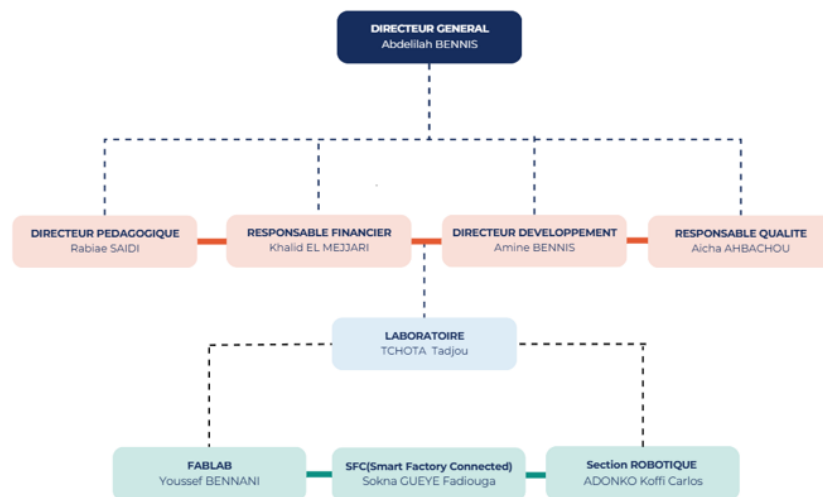


FIGURE 12 – Organigramme de l'entreprise

Annexe 5 : Documentation technique complémentaire

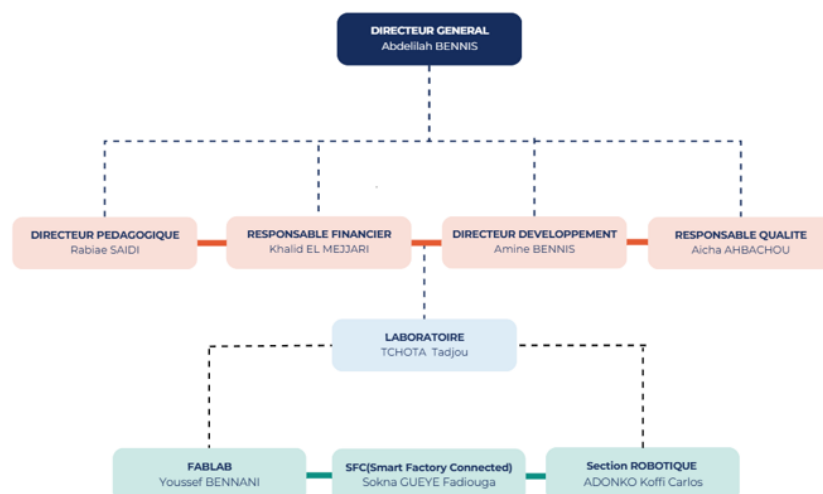


FIGURE 13 – Organigramme de l'entreprise